

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет технологий искусственного интеллекта

Типовик (Задание №4)
по дисциплине "Математический анализ"

Выполнил:
Воронин Илья Андреевич, группа J3112
ИСУ: 372736
Вахменина Татьяна Михайловна J3113
ИСУ: 465374

Проверил:
Ершов А. Р.

г. Санкт-Петербург
2025 г.

Предыстория

Давным-давно, в далекой стране, жили два друга — юный ученый Арнольд и его верный спутник, талантливый художник Леонардо. Они путешествовали по миру, изучая тайны природы и искусства. Однажды, блуждая по густым лесам, они натолкнулись на древнюю башню, окутанную таинственным туманом. Башня выглядела заброшенной, но внутри слышались странные звуки. Друзья решили войти, ведь любопытство всегда было их слабостью. Как только они переступили порог, дверь захлопнулась за ними, и они оказались заперты внутри. — Кажется, мы попали в ловушку! — воскликнул Арнольд. — Но почему? Что здесь такого ценного? — спросил Леонардо, оглядываясь вокруг. Вдруг перед ними появился силуэт старика в длинном плаще. Его голос звучал, словно эхо: — Добро пожаловать в мою башню, смельчаки! Я — великий магистр изображений, и вы стали моими пленниками. Чтобы выбраться отсюда, вам предстоит пройти испытание. Старик махнул рукой, и перед друзьями появились две стопки картинок. Одни были четкими и яркими, другие — размытыми и тусклыми. — Ваша задача проста: определить, кто изображен на каждой картинке, используя свои знания и умения. Но будьте осторожны — некоторые изображения обманчивы! Арнольд и Леонардо переглянулись. Они понимали, что это не просто игра. Их свобода зависела от успеха в этой задаче. Так началось их путешествие по миру изображений, полное загадок и неожиданных открытий.

Введение

Данная лабораторная работа посвящена изучению основ обработки изображений с использованием математических методов и программной реализации на языке Python. В рамках работы будут рассмотрены вопросы представления изображений в цифровом виде, разделения на цветовые каналы, преобразования в оттенки серого, а также применение различных фильтров для улучшения и анализа изображений. Особое внимание будет уделено алгоритму свертки, его применению и выбору методов обработки краевых эффектов.

Основная часть

1. Выбор и загрузка изображения

Для работы было выбрано цветное изображение в формате .jpg.

Исходный код доступен по адресу: https://github.com/crowyy03/ITMO-Laboratory/tree/2_sem/Math%20Anal.



Рис. 1: Исходное цветное изображение.

2. Разделение изображения по цветовым каналам

Цветное изображение в цифровом виде может быть представлено как функция $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$, где каждому пикселю (x, y) ставится в соответствие трехмерный вектор, описывающий его цвет (напри-

мер, в модели RGB). Каждый компонент этого вектора (красный, зеленый, синий) представляет собой двумерный массив, отвечающий за интенсивность данного цветового канала. Пиксельные значения обычно лежат в диапазоне от 0 до 255.

Листинг 1: Код разделения изображения на цветовые каналы

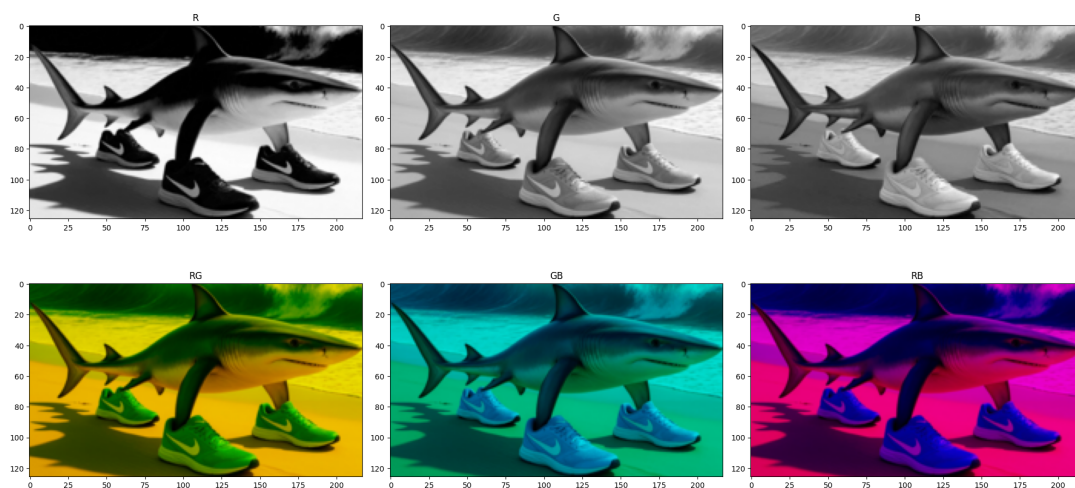
```
from PIL import Image
import numpy as np

img = Image.open('---.jpg')
img_array = np.array(img)

R = img_array[:, :, 0]
G = img_array[:, :, 1]
B = img_array[:, :, 2]
```

3. Визуализация пространственных областей изображения

После разделения изображения на отдельные цветовые каналы (R, G, B), а также создания парных комбинаций (RG, GB, RB), произведена их визуализация. Каждый отдельный канал представляет собой изображение в оттенках серого, где яркость пикселя соответствует интенсивности данного цвета. Парные комбинации позволяют увидеть взаимодействие двух цветовых компонент.



(a) Красный канал (R)

Рис. 2: Визуализация отдельных цветовых каналов и их парных комбинаций.

4. Создание серого изображения

Серое (черно-белое) изображение может быть представлено как функция $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, где каждому пикселю (x, y) соответствует одно значение яркости. Преобразование цветного изображения в серое обычно осуществляется путем взвешенного суммирования интенсивностей цветовых каналов, отражающего чувствительность человеческого глаза к различным цветам. Наиболее распространенная формула для преобразования RGB в оттенки серого: $\text{Gray} = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$

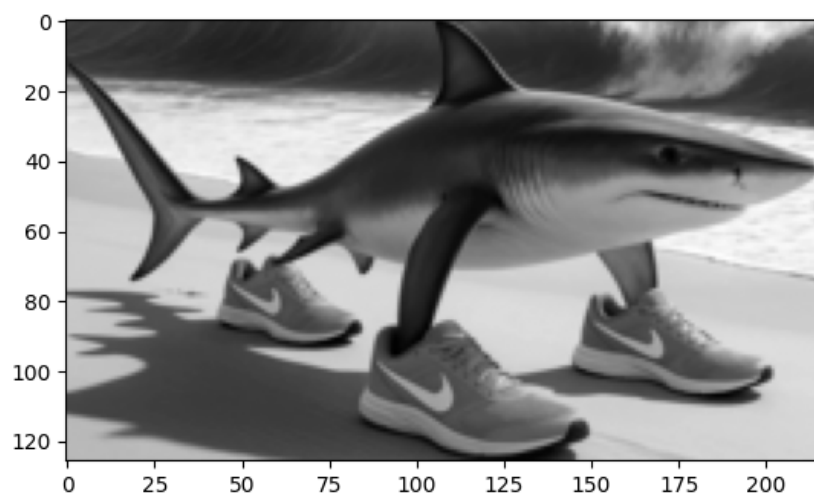


Рис. 3: Серое изображение.

Листинг 2: Код преобразования в серое изображение

```
gray_image = (0.2989 * image[:, :, 0] +
              0.5870 * image[:, :, 1] +
              0.1140 * image[:, :, 2]).astype(np.uint8)

plt.imshow(gray_image, cmap='gray')
```

5. Графики изменения частоты каналов

Для анализа распределения яркости и цветовых интенсивностей, были выбраны несколько строк из матрицы изображения. Построены графики изменения частоты каналов (интенсивности пикселей) для цветного и серого изображений. Для цветных изображений, графики каждого канала (R, G, B) наложены на одну систему координат для сравнения.



Рис. 4: Графики изменения частоты цветовых каналов для выбранной строки.

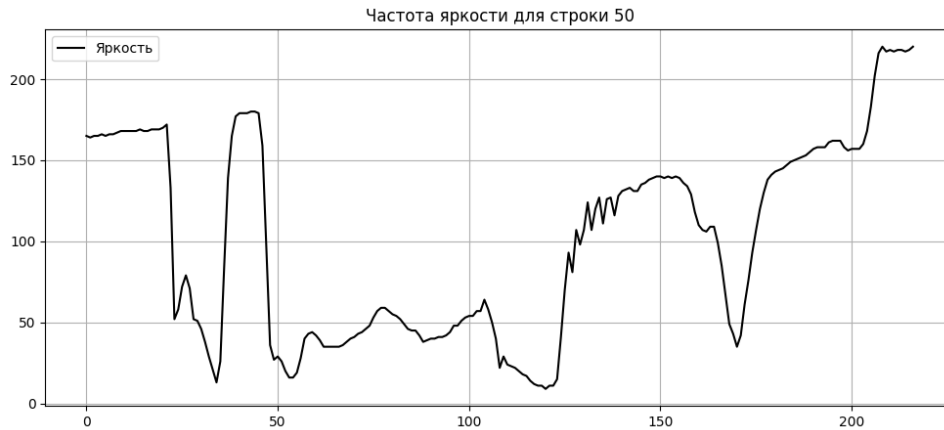


Рис. 5: График изменения частоты для серого изображения для выбранной строки.

6. Свертка функции (Конволюция)

Свертка (или конволюция) — это математическая операция, которая применяется для обработки сигналов и изображений. В контексте изображений, свертка используется для выполнения различных операций, таких как размытие, выделение контуров, повышение резкости и т.д. Формула свертки функции $f(x, y)$ (исходное изображение) с ядром свертки $w(i, j)$ (также называемым фильтром или маской) определяется как: $g(x, y) = w * f(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b w(i, j) \cdot f(x - i, y - j)$ где $g(x, y)$ — изображение после свертки, w — ядро свертки (размером $(2a + 1) \times (2b + 1)$), f — исходное изображение.

Обработка краевых эффектов

При выполнении свертки, особенно на границах изображения, возникает проблема, связанная с тем, что ядро фильтра выходит за пределы изображения. Для обработки этих "невидимых" пикселей используются различные методы:

- **Заполнение нулями (Padding with zeros):** Пиксели за пределами изображения считаются равными нулю. Это может привести к появлению черной рамки вокруг обработанного изображения.
- **Повторение граничных пикселей (Replicate/Clamp):** Крайние пиксели изображения дублируются для заполнения недостающих областей.
- **Зеркальное отражение (Reflect):** Изображение зеркально отражается за пределами границ.
- **Заворачивание (Wrap/Circular):** Изображение рассматривается как циклическое, и пиксели с одной стороны изображения берутся с противоположной стороны.

7. Реализация алгоритма свертки с применением различных фильтров

Реализован алгоритм свертки изображения с использованием различных известных ядер преобразований. Для каждого фильтра предусмотрена возможность изменения входных параметров, если таковые имеются.

а) Пороговая фильтрация (Thresholding)

Пороговая фильтрация используется для сегментации изображения, преобразуя его в бинарное, где каждый пиксель становится либо черным, либо белым в зависимости от того, превышает ли его яркость заданный порог. $g(x, y) = \begin{cases} \text{max_val} & \text{if } f(x, y) > \text{threshold} \\ \text{min_val} & \text{otherwise} \end{cases}$ Данный фильтр не является сверткой в строгом смысле, но часто используется в контексте обработки изображений для бинаризации.

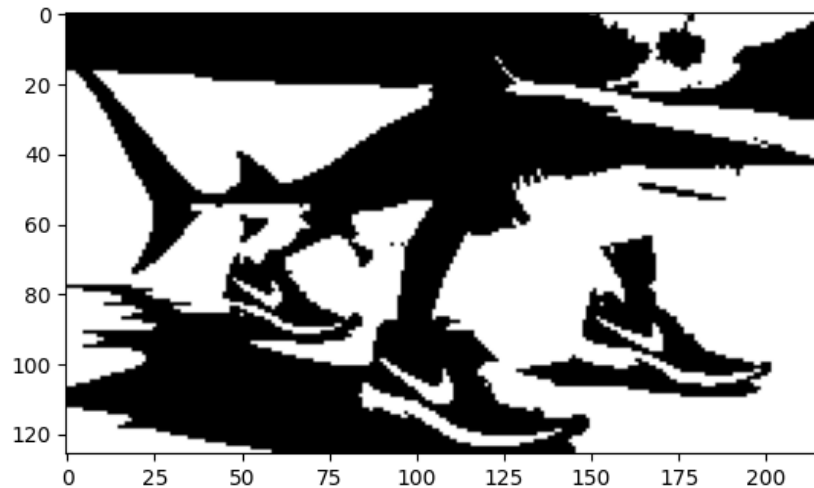


Рис. 6: Изображение после пороговой фильтрации. Порог: [Указать значение].

б) Медианный фильтр (Median Filter)

Медианный фильтр — это нелинейный фильтр, используемый для подавления шума (например, "соль и перец"). Он работает путем замены значения каждого пикселя медианным значением яркостей пикселей в его окрестности (окне фильтра). Медиана менее чувствительна к выбросам, чем среднее арифметическое, что делает его эффективным для сохранения краев изображения.

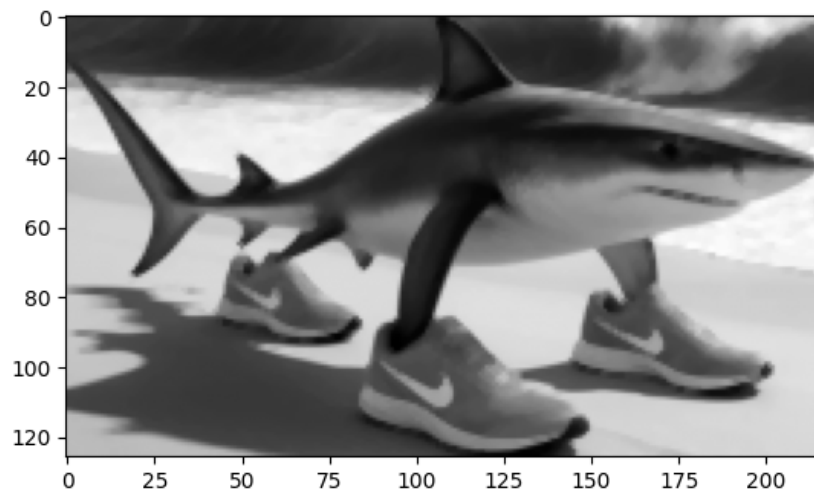


Рис. 7: Изображение после медианной фильтрации.

с) Фильтр Гаусса (Gaussian Filter)

Фильтр Гаусса — это линейный фильтр, который используется для размытия изображения и уменьшения шума. Он основан на функции Гаусса (нормальном распределении), где веса пикселей в ядре свертки распределены по колоколообразной кривой, что придает больший вес центральным пикселям. Параметром фильтра является стандартное отклонение (σ), которое определяет степень размытия. Ядро Гаусса для одномерного случая: $G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$. Двумерное ядро получается умножением двух одномерных ядер.

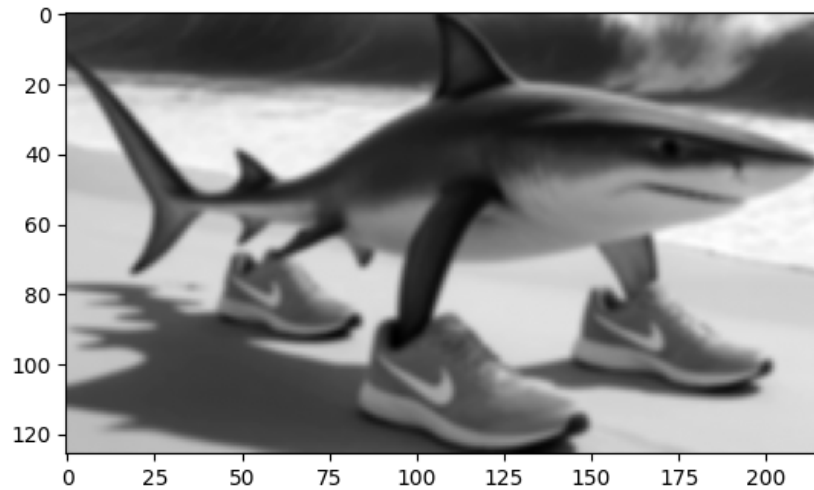


Рис. 8: Изображение после применения фильтра Гаусса.

d) Лине́йный усредня́ющий фильтр (Box Blur)

Лине́йный усредня́ющий фильтр (или усредня́ющий фильтр, box blur) является одним из простейших линейных фильтров для размытия изображения. Он работает путем замены значения каждого пикселя средним арифметическим значением яркостей пикселей в его окрестности (окне фильтра). Все пиксели в окне имеют одинаковый вес. Ядро фильтра представляет собой матрицу из единиц, нормированную так, чтобы сумма всех элементов ядра равнялась 1. Например, для ядра 3×3 :

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

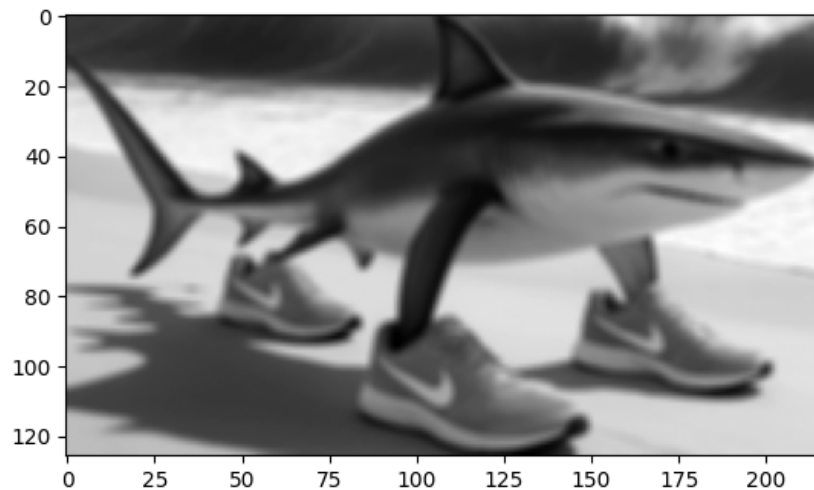


Рис. 9: Изображение после применения линейного усредняющего фильтра.

е) Фильтр Собеля (Sobel Filter)

Фильтр Собеля — это фильтр обнаружения краев, который вычисляет приближение градиента интенсивности изображения. Он использует два ядра 3×3 : одно для горизонтальных изменений (по оси X) и одно для вертикальных изменений (по оси Y). Ядро Собеля для горизонтальных изменений

$$(G_x): G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \quad \text{Ядро Собеля для вертикальных изменений } (G_y): G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

Величина градиента в каждой точке вычисляется как $M = \sqrt{G_x^2 + G_y^2}$.

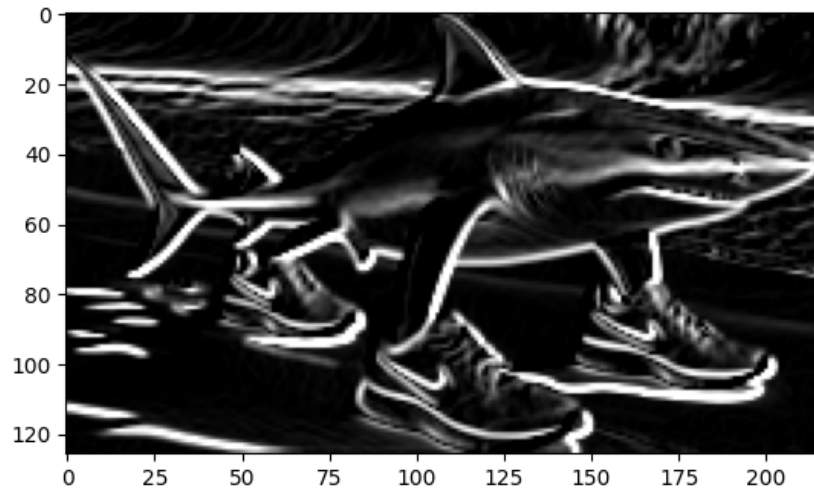
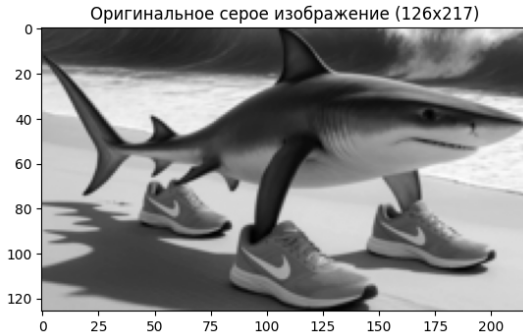


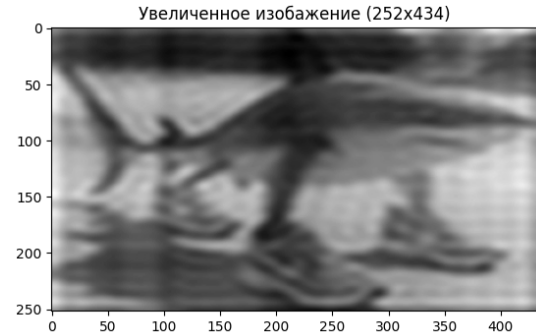
Рис. 10: Изображение после применения фильтра Собеля.

8. Увеличение изображения

Алгоритм увеличения изображения (масштабирования) позволяет изменить разрешение изображения, сохраняя при этом его качество. Основная задача при увеличении — интерполировать значения новых пикселей, которые не существуют в исходном изображении. В данной работе была реализована методика, описанная в статье [Укажите ссылку на статью, если она не была предоставлена]. Реализация может использовать как методы численного интегрирования (например, для построения интерполяционной функции), так и встроенные алгоритмы (например, билинейная или бикубическая интерполяция).



(a) Исходное изображение



(b) Увеличенное изображение

Рис. 11: Сравнение исходного и увеличенного изображения. Алгоритм: [Указать алгоритм], Коэффициент масштабирования: [Указать коэффициент].

Листинг 3: Код увеличения изображения

```
def compute_fourier_coefficients_manual(image_channel, m_max, n_max):
    l1, l2 = image_channel.shape[1], image_channel.shape[0]

    ajk_coefficients = np.zeros((m_max + 1, n_max + 1), dtype=np.float32)

    total_j_iterations = m_max + 1

    for j in tqdm(range(total_j_iterations), desc="ajk"):
        for k in range(n_max + 1):
            sum_val = 0.0
            for x_idx in range(l1):
```

```

        for y_idx in range(12):
            f_xy = image_channel[y_idx, x_idx]

            cos_arg_x = j * np.pi * x_idx / 11
            cos_arg_y = k * np.pi * y_idx / 12

            sum_val += f_xy * np.cos(cos_arg_x) * np.cos(cos_arg_y)

        ajk_coefficients[j, k] = sum_val / (11 * 12)

    return ajk_coefficients

def reconstruct_and_upscale_fourier_series(ajk_coefficients, original_dimensions, target
m_max, n_max = ajk_coefficients.shape[0] - 1, ajk_coefficients.shape[1] - 1
original_height, original_width = original_dimensions[0], original_dimensions[1]

upscaled_image = np.zeros((target_height, target_width), dtype=np.float32)

for y_prime in tqdm(range(target_height), desc="
    for x_prime in range(target_width):
        sum_val = 0.0

        x_scaled = x_prime * original_width / target_width
        y_scaled = y_prime * original_height / target_height

        for j in range(m_max + 1):
            for k in range(n_max + 1):
                ajk = ajk_coefficients[j, k]

                cos_arg_x = j * np.pi * x_scaled / original_width
                cos_arg_y = k * np.pi * y_scaled / original_height

                sum_val += ajk * np.cos(cos_arg_x) * np.cos(cos_arg_y)

        upscaled_image[y_prime, x_prime] = sum_val

    return np.clip(upscaled_image, 0, 255).astype(np.uint8)

```

9. Дополнительно: Фурье-спектр изображения

Фурье-преобразование является мощным инструментом анализа изображений, позволяющим перейти из пространственной области в частотную. Фурье-спектр изображения показывает распределение частотных компонент: низкие частоты соответствуют общим характеристикам яркости и крупным структурам, высокие частоты — мелким деталям и краям.

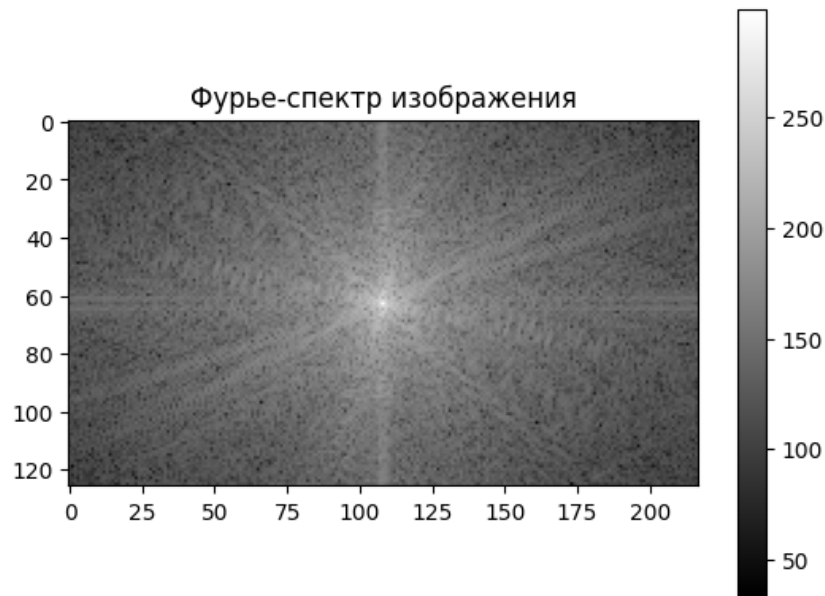


Рис. 12: Фурье-спектр изображения.

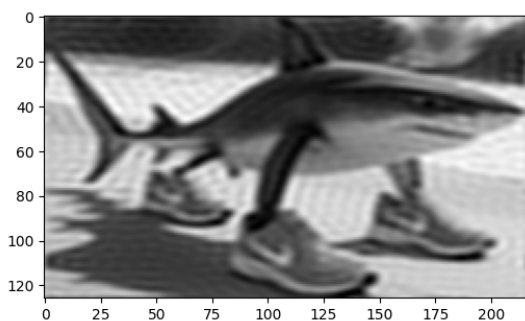
10. Дополнительно: Высокочастотная и низкочастотная фильтрация

Фильтрация в частотной области позволяет выборочно усиливать или подавлять определенные частотные компоненты изображения.

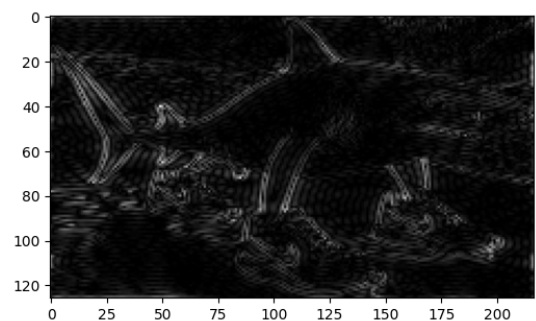
- **Низкочастотная фильтрация:** Устраняет высокочастотные компоненты (шум, мелкие детали), что приводит к размытию изображения.
- **Высокочастотная фильтрация:** Устраняет низкочастотные компоненты (общие изменения яркости), выделяя края и детали изображения. Это может быть использовано для повышения резкости.

Путем удаления низкочастотной составляющей, можно получить изображение с повышенной резкостью, которое содержит края.

Комментарий для вставки: Вставьте изображение после низкочастотной фильтрации и изображение после высокочастотной фильтрации/повышения резкости.



(а) Изображение после низкочастотной фильтрации.



(б) Изображение после высокочастотной фильтрации (повышение резкости).

Рис. 13: Результаты частотной фильтрации.

Заключение

В ходе выполнения данной лабораторной работы были освоены базовые принципы цифровой обработки изображений, включая их представление, работу с цветовыми каналами, преобразование в

оттенки серого, а также применение различных пространственных и частотных фильтров. Реализованные алгоритмы позволили наглядно продемонстрировать влияние различных преобразований на качество и характеристики изображения, а также закрепить навыки работы с библиотеками `numpy`, `PIL`, `matplotlib` (или `plotly`).